Islamic University of Technology

Organization of Islamic Cooperation



Name	**:** Antara Arifa Mullick
ID	**:** 210042132
Program	**:** SWE
Department	**:** CSE
Subject	**:** Artificial Intelligence
Subject Code	**:** CSE 4618
Title	**:** Markov Decision Process
Date of Submission	**:** 18th August, 2025

1. Introduction

This lab focuses on implementing value iteration algorithms to solve Markov Decision Processes (MDPs) in a Gridworld environment. The primary goal was to develop and compare three variants of the value iteration algorithm:

- Standard Batch Value Iteration
- Asynchronous Value Iteration
- Prioritized Sweeping Value Iteration

Additionally, the lab involved understanding how different MDP parameters such as the discount factor, noise, and living reward influence the optimal policies of agents navigating a grid environment with rewards, penalties, and stochastic transitions. The objective was to demonstrate key concepts in reinforcement learning and dynamic programming.

2. Problem Analysis

Core Challenges

1. **Value Iteration Implementation**: The standard value iteration algorithm requires careful attention to "batch" updates, where all state values are computed based on the previous iteration's values, rather than being updated in place.

2. **Policy Extraction**: Extracting the optimal policy involves finding the action that maximizes expected future rewards, taking into account transition probabilities and the discount factor.

3. **Parameter Sensitivity Analysis**: Understanding the interaction between different MDP parameters such as the discount factor, noise, and living reward, and how these factors affect the agent's behavior and decision-making.

4. **Asynchronous Updates**: Implementing state updates where only one state is updated per iteration. This approach requires careful indexing and handling of terminal states.

5. **Prioritized Sweeping**: Developing an efficient strategy that updates states based on the magnitude of potential value changes, allowing computational

effort to focus on the most impactful states.

Unique Algorithmic Aspects

The lab required the implementation of three distinct value iteration variants:

- **Batch Value Iteration**: Updates all states simultaneously using the previous iteration's values.
- **Asynchronous Value Iteration**: Updates one state per iteration, cycling through all states.
- **Prioritized Sweeping**: Updates states based on the priority determined by the magnitude of value changes.

3. Solution Explanation

Question 1: Standard Value Iteration

The Standard Value Iteration follows the Bellman equation, which computes the value of each state iteratively until convergence. The algorithm updates all states simultaneously using the previous iteration’s values.

```
def runValueIteration(self):
    for i in range(self.iterations):
        newValues = self.values.copy()
        for state in self.mdp.getStates():
            if self.mdp.isTerminal(state):
                continue
            actions = self.mdp.getPossibleActions(state)
            if not actions:
                continue
                q_values = [self.computeQValueFromValues(state, action) for action in actions]
```

```
            newValues[state] = max(q_values)
        self.values = newValues
```

Key Design Decisions:

- Used `newValues.copy()` to ensure batch updates and prevent in-place modifications.
- Handled terminal states and states with no possible actions.
- Separated Q-value computation into a dedicated method for reusability.

Complexity Analysis:
The time complexity is O(I × S × A × S'), where:

- I is the number of iterations
- S is the number of states
- A is the number of actions
- S' is the set of successor states.

Question 2: Bridge Crossing Analysis

In the bridge crossing scenario, the challenge was to find parameters that encourage the agent to cross a narrow bridge despite the risk. The solution involved setting the noise to 0 and maintaining the default discount factor of 0.9, which made the risky but shorter path more attractive.

```
def question2():
    answerDiscount = 0.9
    answerNoise = 0
    return answerDiscount, answerNoise
```

Rationale:
Setting the noise to 0 ensures deterministic actions, encouraging the agent to take the high-reward but risky path to the terminal state (+10), rather than staying near the low-reward state (+1).

Question 3: Policy Analysis

This question required finding parameter combinations that produced five distinct behavioral patterns. The parameters such as discount, noise, and living reward were adjusted for each scenario:

- **3a: Close Exit, Risk Cliff**
  Parameters: discount=0.2, noise=0.0, livingReward=-1.0
  Logic: Low discount makes immediate rewards more valuable, and negative living rewards encourage quick termination.

- **3b: Close Exit, Avoid Cliff**
  Parameters: discount=0.3, noise=0.3, livingReward=-0.1
  Logic: Moderate noise makes risky paths less reliable, encouraging safer routes.

- **3c: Distant Exit, Risk Cliff**
  Parameters: discount=0.9, noise=0.0, livingReward=-0.2
  Logic: High discount values future rewards, enabling precise risky navigation.

- **3d: Distant Exit, Avoid Cliff**
  Parameters: discount=0.9, noise=0.3, livingReward=-0.2
  Logic: High discount with noise forces safer long-term planning.

- **3e: Never Terminate**
  Parameters: discount=1.0, noise=0.0, livingReward=1.0
  Logic: Positive living reward prevents termination, encouraging indefinite exploration.

Question 4: Asynchronous Value Iteration

The Asynchronous Value Iteration approach updates only one state per iteration in a cyclic manner, allowing more memory-efficient updates.

```
def runValueIteration(self):
```

```
    states = self.mdp.getStates()
    for i in range(self.iterations):
        state = states[i % len(states)]
        if self.mdp.isTerminal(state):
            continue
        actions = self.mdp.getPossibleActions(state)
        if not actions:
            continue
        q_values = [self.computeQValueFromValues(state, action) for action in actions]
        self.values[state] = max(q_values)
```

Advantages:

- More memory-efficient than batch updates.
- Can converge faster in some scenarios.

Complexity:
The time complexity per iteration is O(I × A × S'), where I is the number of iterations and S' is the set of successor states.

4. Findings and Insights

Performance Observations

- Convergence Speed: Prioritized sweeping converged faster than standard value iteration, especially in sparse reward environments where most states have similar values.
- Parameter Sensitivity: Small changes in noise and discount factors can dramatically alter optimal policies, emphasizing the need for careful parameter

tuning.

- Policy Stability: Asynchronous updates sometimes showed more stable intermediate policies compared to batch updates.

Behavioral Patterns

- Risk-Taking: Low noise encourages risky behaviors, while high noise promotes conservative strategies.

- Time Preference: The discount factor strongly influences whether agents prefer immediate or delayed rewards.

- Living Costs: Negative living rewards create urgency, while positive rewards encourage exploration.

5. Challenges Faced

Implementation Challenges

1. **Batch vs Online Updates**: Implementing in-place updates initially caused incorrect convergence, resolved by creating separate value containers for each iteration.

2. **Terminal State Handling**: Ensured proper skipping of terminal states while maintaining the correct cycling of states in asynchronous updates.

3. **Priority Queue Management**: Managed priority queue updates carefully to avoid duplicates and ensure efficient computation.

Parameter Tuning Challenges

Finding the right parameter combinations for the given scenarios was an iterative process. The GUI visualization was used to verify the policy's behavior at each stage.

6. Hyperparameter Exploration

Discount Factor Impact

- High values (0.9+) encourage long-term planning, whereas low values (0.1-0.3) promote quick termination.
- A discount factor of 1.0 can lead to infinite exploration when combined with positive living rewards.

Noise Level Effects

- **Zero noise**: Enables deterministic actions.
- **Moderate noise**: Forces the agent to account for uncertainty and promotes robust policies.
- **High noise**: Makes risky strategies ineffective.

Living Reward Influence

- **Negative values**: Encourage quick termination.
- **Positive values**: Prevent termination entirely, encouraging infinite exploration.

7. Additional Insights

Algorithmic Efficiency

- Prioritized sweeping showed promising efficiency, particularly in large state spaces where uniform updates waste computational resources.

Real-World Applications

- Safety-critical systems may benefit from higher noise levels to ensure robust policies.

- Time-sensitive applications require careful discount factor tuning to optimize decision-making speed.

8. Conclusion

This lab provided valuable hands-on experience with Markov Decision Processes (MDPs) and various value iteration algorithms, including batch value iteration, asynchronous value iteration, and prioritized sweeping. By implementing these algorithms, we were able to explore how MDP parameters such as discount factors, noise, and living rewards influence the optimal policies in a dynamic environment.